

2026 年春季学期
计算学部《软件工程》课程

Lab 2 项目计划与原型设计 实验报告

姓名	学号	班级	课程名称
刘宇轩	2023112455	23R0313	软件工程

目 录

- 1 实验要求
- 2 项目概述
- 3 用户故事
- 4 用户故事的工作量估算
- 5 迭代计划
- 6 使用 CodeArts 或其他工具管理用户故事和迭代计划
- 7 使用大模型辅助生成用户故事和迭代计划
- 8 原型设计
- 9 计划与实际进度
- 10 小结

1 实验要求

这次 Lab2 主要要求我们围绕一个具体项目完成项目计划和原型设计，包括梳理项目概述、整理用户故事、估算工作量、安排迭代计划，并用项目管理工具展示需求管理结果，同时完成重点用户故事的原型设计。结合我这次做的 CodePilot 项目，我把项目文档、CodeArts 里的 Story 列表，以及真实运行截图一起整理进这份报告里。

2 项目概述

我这次选择的项目是 CodePilot：从零实现的智能 Coding Agent 开发平台。简单来说，它不是普通的聊天机器人，而是一个能接收自然语言任务、理解仓库结构、先给出执行计划，再去修改代码、运行命令和验证结果的原型系统。

项目的目标用户主要是课程实验中的开发者、学生程序员和个人开发者。这个项目最有价值的地方在于：它强调“先计划、再执行、能验证、可回退”，这样做出来的系统更像真正的软件工程工具，而不是只会给建议的对话模型。

3 用户故事

3.1 用户故事清单及优先级

用户故事编号	用户故事简称	用户故事描述	优先级估算	上游故事编号
US-01	自然语言输入任务	作为开发者，我希望直接用自然语言描述任务，这样系统就能先帮我整理出规范化请求。	5	-
US-02	仓库扫描与上下文理解	作为开发者，我希望系统自动扫描仓库结构和关键文件，方便它理解当前项目。	5	US-01
US-03	任务规划展示	作为开发者，我希望系统先展示执行计划和风险提示，再决定是否执行。	5	US-01,US-02
US-04	多文件代码修改	作为开发者，我希望系统能修改多个文件，而不是只给文字建议。	4	US-03
US-05	命令执行与测试验证	作为开发者，我希望系统能跑 pytest、ruff 这些命令，帮我判断任务是否真正	4	US-03,US-04

		完成。		
US-06	代码变更对比	作为开发者，我希望看到修改前后的差异，方便检查结果。	4	US-04,US-05
US-07	危险操作确认	作为开发者，我希望高风险命令能被识别并确认，避免误操作。	4	US-03
US-08	执行日志记录	作为开发者，我希望系统保留执行日志，方便复盘。	3	US-05
US-09	失败修复与重试	作为开发者，我希望测试失败时系统能给出修复建议并支持再次尝试。	3	US-05
US-10	历史会话与任务记录	作为开发者，我希望保留历史任务记录，便于查找之前做过什么。	3	US-08
US-11	执行边界控制	作为开发者，我希望系统只在指定范围内执行，保证安全。	3	US-03
US-12	一键回退	作为开发者，我希望出问题能快速回退到之前快照。	3	US-08

3.2 用户故事 1

- 作为一个开发者，我想要直接输入自然语言任务，并选择 plan 或 auto 模式，这样系统就能先理解我的目标。
- 这个故事的商业价值在于降低使用门槛，不需要先背复杂命令。
- 如果输入为空、路径错误或者模式不合法，系统应该直接报错。

3.3 用户故事 2

- 作为一个开发者，我希望系统先扫描仓库结构、README、配置文件和测试入口，再去规划任务。
- 这样能减少人工找文件的时间，也能让后续计划更靠谱。
- 如果关键文件缺失，系统也应该明确提示。

3.4 用户故事 n

- 我这里选择 US-05 作为代表。作为一个开发者，我希望系统执行 `pytest`、`ruff` 等命令，并把结果和退出码保存下来。
- 这样做的好处是结果可验证，不会停留在“看起来像做完了”。
- 如果测试失败，系统应该给出失败提示，而不是直接跳过。

4 用户故事的工作量估算

这部分我主要参考了故事复杂度和实现依赖关系。因为这次更接近个人原型开发，所以三轮估算我理解成“初估—复核—最终确认”。单位按小时统计。

故事编号	故事简称	第一轮	第二轮	第三轮	最终估算
US-01	任务输入	3	2	2	2
US-02	仓库理解	5	5	5	5
US-03	计划展示	5	5	5	5
US-04	代码修改	8	8	8	8
US-05	命令执行	8	8	8	8
US-06	变更对比	5	3	3	3
US-07	风险确认	5	3	3	3
US-08	日志记录	3	3	3	3
US-09	失败修复	5	5	5	5
US-10	历史会话	5	5	5	5
US-11	边界控制	3	3	3	3
US-12	一键回退	3	3	3	3

5 迭代计划

我把项目分成两轮迭代。第一轮先把主流程打通，第二轮再补日志、回退和鲁棒性。这样安排的原因是：如果主链路不通，后面的增强功能意义也不大。

Sprint 1（第 1-6 周）主要做：自然语言任务输入、仓库理解、计划展示、多文件修改、命令执行和风险确认。

Sprint 2（第 7-8 周）主要做：执行日志、失败修复、历史会话、执行边界和一键回退。

Sprint	时间安排	主要内容	验收标准
Sprint 1	第 1-6 周	任务输入、仓库扫描、Plan 展示、代码修改、命令执行、风险确认	至少能跑通一条从输入到验证的闭环
Sprint 2	第 7-8 周	日志记录、失败修复、历史会话、执行边界、一键回退	能追踪 history/logs，并支持 snapshot 恢复

6 使用 CodeArts 或其他工具管理用户故事和迭代计划

这一部分我结合了真实的 CodeArts 截图来写，而不是完全用示意图代替。截图里能看到我已经把 CodePilot 的主要用户故事录入到 Story 列表中。

6.1 建立团队、项目、成员

这张截图没有拍到 CodeArts 顶部导航，所以团队名和项目名看不清。但从右侧成员相关字段里可以看到账号 lyx20050g，说明这些 Story 已经录入到项目管理系统中。

70777980	Story	US-12 一键回退	--	新建	lyx20050g	☆	🔍	⋮
70777979	Story	US-11 执行边界控制	--	新建	lyx20050g	☆	🔍	⋮
70777978	Story	US-10 历史会话与任务记录	--	新建	lyx20050g	☆	🔍	⋮
70777977	Story	US-09 失败修复与重试	--	新建	lyx20050g	☆	🔍	⋮
70777975	Story	US-08 执行日志记录	--	新建	lyx20050g	☆	🔍	⋮
70777918	Story	US-07 危险操作确认	--	新建	lyx20050g	☆	🔍	⋮
70777917	Story	US-06 代码变更对比	--	新建	lyx20050g	☆	🔍	⋮
70777916	Story	US-05 命令执行与测试验证	--	新建	lyx20050g	☆	🔍	⋮
70777915	Story	US-04 多文件代码修改	--	新建	lyx20050g	☆	🔍	⋮
70777914	Story	US-03 任务规划展示	--	新建	lyx20050g	☆	🔍	⋮
70777913	Story	US-02 仓库扫描与上下文理解	--	新建	lyx20050g	☆	🔍	⋮
70777912	Story	US-01 自然语言输入任务	--	新建	lyx20050g	☆	🔍	⋮

图：CodeArts Story 列表截图

6.2 Product Backlog（全部用户故事）

从截图中可以直接识别出 US-01 到 US-12 共 12 条 Story，内容包括自然语言输入任务、仓库扫描与上下文理解、任务规划展示、多文件代码修改、命令执行与测试验证、代码变更对比、危险操作确认、执行日志记录、失败修复与重试、历史会话与任务记录、执行边界控制、一键回退。当前这些 Story 的状态都显示为“新建”。

6.3 Sprint Planning（规划迭代）

虽然这张截图不是迭代页面，但结合项目文档中的迭代安排，我把高优先级故事（US-01~US-07）放到第一轮，日志、失败修复和回退相关故事放到第二轮，这样更符合“先核心、后增强”的开发顺序。

6.4 形成每个 Sprint 的 Story Board

这张图本身是列表页，不是看板页，所以没有直接显示“待办、进行中、已完成”三列。这里我在分析时保留了这个事实，没有硬编看板状态。

6.5 形成每个 Sprint 的 Burndown Chart

当前上传的截图里也没有燃尽图，所以我没法从图中提取日期和燃尽趋势。后续如果补充了迭代页或燃尽图截图，这一节可以再替换成更完整的内容。

7 使用大模型辅助生成用户故事和迭代计划

这次我确实用到了大模型，但主要是让它帮我做“初稿整理”和“表达优化”，不是直接替我决定技术方案。

- 提示词 1：请根据 CodePilot 项目目标，生成 8~12 个用户故事，并按优先级排序。
- 提示词 2：请根据 Plan-Execute + TDD 的思路，把这些故事分配到两轮 Sprint。
- 提示词 3：请把技术文档改写成适合软件工程课程实验报告的说明文字。

我觉得大模型在这一步最大的作用是节省了组织语言的时间，但最终哪些故事优先、怎么分 Sprint、哪些内容能写进报告，还是要自己核对项目文档和实际截图。

8 原型设计

原型设计这部分我重点围绕任务输入、仓库理解、计划确认、命令执行和历史日志几个高优先级故事来做。为了让报告内容和真实项目更一致，我还插入了一张真实运行截图，作为原型和实际效果之间的对应说明。

8.1 大模型生成的原型

这个页面更像项目首页，主要展示任务输入入口和最近会话。



8.2 用户故事 1

这里重点体现任务描述、工作目录和 plan/auto 模式切换。



8.3 用户故事 2

这里主要体现仓库扫描结果、关键文件和摘要输出。



8.4 用户故事 3

这里展示 Plan 阶段应该包含的 summary、steps、风险提示和确认按钮。



8.5 用户故事 4

这里体现命令执行、测试结果和失败提示。



8.6 用户故事 5

这里体现 history、logs、snapshot 等追踪能力。



下面这张图是我结合项目运行过程加入的真实截图，可以看出 CodePilot 已经能在真实示例仓库里生成 plan 状态、执行 `pytest/ruff`、记录 `history/logs`，并给出 `failure hints`。

CodePilot 真实情景测试

场景：在一个真实创建的示例仓库中运行 auto 模式，故意保留失败测试，以验证执行、失败提示、历史与日志链路。

1. 运行过程

```
$ /home/hermes/codepilot-coding-agent/.venv/bin/python -m codepilot.cli run --workdir /home/hermes/codepilot-coding-agent/artifacts/real-scenario/demo_repo --mode auto 验证失败测试并给出修复建议
session_id: 4fa8003c50bf
status: ready_to_execute
summary: Generated a plan for '验证失败测试并给出修复建议' using 7 core tools. Current mode: auto.
steps:
- Scan repository structure
- Read key project files
- Prepare an execution plan
- Review risks and seek confirmation
risk: low (未检测到显著风险关键词)
command: pytest -q => 1
command: ruff check . => 0
failure_hints:
- pytest 未通过：先查看失败用例。断言差异和测试夹具是否与当前实现一致。
- 检测到断言失败：优先核对预期输出、边界值和测试数据，而不是直接绕过测试。
rollback_snapshot: snapshot-20260416142215431144
```

2. 历史记录

```
$ /home/hermes/codepilot-coding-agent/.venv/bin/python -m codepilot.cli history --workdir /home/hermes/codepilot-coding-agent/artifacts/real-scenario/demo_repo
4fa8003c50bf    pending_review 验证失败测试并给出修复建议
```

3. 会话日志

```
$ /home/hermes/codepilot-coding-agent/.venv/bin/python -m codepilot.cli logs 4fa8003c50bf --workdir /home/hermes/codepilot-coding-agent/artifacts/real-scenario/demo_repo
plan_status=ready_to_execute
risk=low:未检测到显著风险关键词
snapshot=snapshot-20260416142215431144
command=pytest -q exit=1
command=ruff check . exit=0
hint=pytest 未通过：先查看失败用例。断言差异和测试夹具是否与当前实现一致。
hint=检测到断言失败：优先核对预期输出、边界值和测试数据，而不是直接绕过测试。
```

9 计划与实际进度

任务名称	计划时间长度（分钟）	实际耗费时间（分钟）	提前或延期的原因分析
用户故事整理	90	80	因为项目文档本身比较完整，所以整理速度比预期快一些
迭代计划安排	60	70	需要把技术路线和课程模板对应起来，所以多花了一点时间
CodeArts 内容整理	40	55	截图是列表页，不是迭代页，需要额外说明哪些信息能确认、哪些不能确认
原型草图设计	120	130	为了保证报告完整，

			我补了 5 个高优先级原型
报告整合与排版	90	110	要同时处理文档、截图、表格和模板格式，排版时间比较长

整体来看，项目的实际进度和原来的迭代计划基本一致。最重要的主线功能已经能展示出来，差的主要是 CodeArts 迭代页和燃尽图这类过程截图，如果后续继续完善，可以把这部分补全。

10 小结

做完这次 Lab2 之后，我的感觉是：大模型确实能提高整理材料和生成初稿的效率，但真正决定报告质量的，还是自己有没有把项目内容、截图证据和实验要求对应起来。对 CodePilot 这种项目来说，最关键的不是把话说得多漂亮，而是有没有把“计划—执行—验证—回退”这条主线讲清楚。

另外，这次我也发现项目管理截图很重要。真实截图能证明需求已经落到了工具里，而不是只停留在文档层面。后面如果再继续做第 2 轮检查，我应该把看板页、迭代页和燃尽图也补齐，这样报告会更完整。